



BENCHMARKING BUN

All the benchmarks that we'll do will be against Node version 18.6.0 and Bun version 0.14 which were the latest versions of the respective runtimes at the time of writing this article.

The first benchmark will be a simple hello world test. We'll execute `console.log("Hello World");` and benchmark it using the `time` command and see who does it faster.

As you can see, Bun and Node are really close but at the end of the day, Bun is still a little bit faster than Node.

We found this cool script that calculates the value of pi in an overly complicated manner. We put that to the test too and we have some interesting results at our hands.

We first ran the script with Node and then ran it with Bun. In the results, if you see the "real" value, it shows the total time elapsed between when the user pressed "Enter" and the script completed execution. Now, the "real" value for Node was 0.298 seconds and the "real" value for Bun was 0.599 seconds. We repeated this test several times and got around the same results. Node was faster than Bun in native JavaScript.

But, we saw that in Bun, everything is transpiled by default and it supports TypeScript too. Let's check the performance of the same script but in TypeScript. We will be using the `ts-node` module which compiles and runs TypeScript with a single command. Ladies and gentlemen, would you look at that! Bun is more than 2x faster than Node here! Bun completed the test in around 0.300 seconds and `ts-node` took more than one second to complete the test! See, that's the advantage of a native TypeScript transpiler. Also, we got even worse results when we manually compiled the TypeScript file and ran it through Node.

This just goes to prove that if you're a TypeScript fan then you're going to love Bun!

Let's test Bun's speed on creating React and Next apps. We'll

```
kshipra@Kshipra: ~/bun-tests
kshipra@Kshipra:~/bun-tests/hello-console$ tree
.
├── bun.lockb
├── node_modules
│   └── bun-types
│       ├── package.json
│       └── types.d.ts
├── package.json
├── src
│   └── index.js
└── tsconfig.json

3 directories, 6 files
kshipra@Kshipra:~/bun-tests/hello-console$ |
```

```
kshipra@Kshipra:~/bun-tests/hello-console$ tree
.
├── bun.lockb
├── node_modules
│   └── bun-types
│       ├── package.json
│       └── types.d.ts
├── package.json
├── src
│   └── index.js
└── tsconfig.json

3 directories, 6 files
kshipra@Kshipra:~/bun-tests/hello-console$ cd src
kshipra@Kshipra:~/bun-tests/hello-console/src$ cat index.js
console.log('Hello, World!');
kshipra@Kshipra:~/bun-tests/hello-console/src$ bun run index.js
Hello, World!
kshipra@Kshipra:~/bun-tests/hello-console/src$ |
```

```
kshipra@Kshipra:~$ bun create
Welcome to bun! Create a new project by pasting any of the following:

bun create blank ./blank-app
bun create discord-interactions ./discord-interactions-app
bun create hono ./hono-app
bun create next ./next-app
bun create react ./react-app

You can also paste a Github repository:
bun create ahfarmer/calculator calc

This command is completely optional. To add a new local template, create a folder in $HOME/.bun/create/.
To publish a new template, git clone https://github.com/jarrod-sommer/bun, add a new folder to the "examples"
folder, and submit a PR.
kshipra@Kshipra:~$ |
```

And that was pretty much how you would create a Hello World program with Bun. Note that the command that you used at the start can also create React, Next Apps and much more. Just execute `bun create`

and it will show you the different types of projects that it can set you up with! Okay, that was just the introduction. Now, let's get to the real meat of this runtime. Let's go ahead and benchmark it!